

Approach Document

C Programming Assistant Chatbot

Code Generation · Code Analysis · Debugging · Q&A

1. Objective

Build a chatbot on top of an existing large language model that can perform four distinct tasks for C programming: (1) analyze C code, (2) debug C code, (3) generate C code, and (4) answer general C-language questions. This document outlines the recommended architecture and reasoning — specifically, why we use an existing LLM combined with RAG and tool integration, rather than training a model from scratch or relying on full fine-tuning.

2. Build vs. Fine-tune vs. RAG — Decision Summary

Option	Verdict	Reason
Train an LLM from scratch	Rejected	Costs millions of dollars in compute and data. Existing models already understand C well from pretraining.
Full fine-tuning as primary approach	Not recommended (for now)	Needs a large labeled dataset, expensive to run, and must be redone whenever behavior needs to change. RAG updates instantly by just editing documents.
Existing LLM + RAG + Tool Use	Recommended	Combines the LLM's existing reasoning with grounded retrieval and real tool output (compiler / static analyzer) for reliable results.
Light fine-tuning (LoRA) later	Optional polish step	Only if output format/tone is still consistently wrong after prompting + RAG have been fully tried.

3. Task-by-Task Breakdown

The four required capabilities do not all need the same technique. Each is mapped to its best-fit approach below.

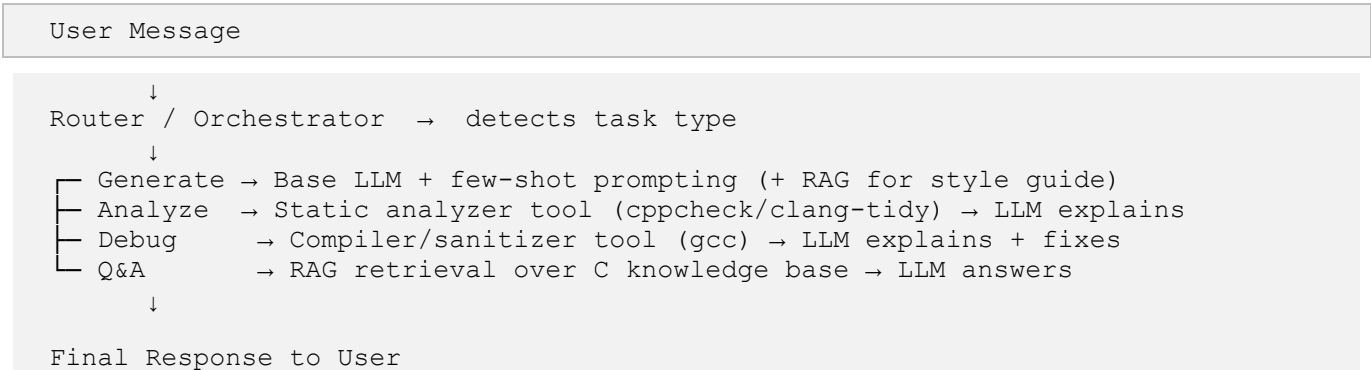
Task	Best Approach	Why
C Code Generator	Base LLM + strong prompting	Modern LLMs already generate C well. RAG only needed to enforce our own style guide / internal libraries.
C Code Analyzer	LLM + Tool Use (cppcheck, clang-tidy)	LLM reasoning alone can miss real bugs. Static analysis tools catch memory leaks / undefined behavior reliably; LLM explains results in plain language.
C Code Debugger	LLM + Tool Use (gcc, sanitizers) + light RAG	Real compiler/sanitizer output is far more reliable than the LLM guessing. RAG over past bug patterns adds extra context.
C Question Answering	RAG over curated C knowledge base	Classic RAG use case — grounds answers in authoritative reference material (language standard, textbooks, course docs) instead of model memory alone.

4. Recommended Architecture

Five layers, working together:

1	Base LLM Handles reasoning, explanation, and generation (Claude / GPT / open-source code model via API).
2	RAG Layer Curated C knowledge base (language reference, common pitfalls, internal docs) feeds grounded context for Q&A and style-guide adherence.
3	Tool-Use Layer Function-calling access to a real compiler (gcc) and static analyzer (cppcheck / clang-tidy) so analyze/debug tasks use real signal, not guesswork.
4	Router / Orchestrator Detects which of the 4 tasks the user wants and dispatches to the right prompt + tool combination.
5	(Optional, later) Fine-tuning Light LoRA fine-tuning only if output format/tone is still off after prompting + RAG + tools are fully utilized.

5. Architecture Flow (Text Diagram)



6. Implementation Roadmap

Phase	Deliverable	Details
Phase 1	Base chatbot + routing	Connect to an existing LLM API. Build a router/classifier that detects which of the 4 tasks the user wants.
Phase 2	RAG knowledge base for Q&A	Curate C reference material (language standard, textbook excerpts, course docs). Build ingestion, chunking, embedding, vector store, retrieval pipeline.
Phase 3	Tool-use layer for analyze/debug	Integrate gcc (compile + run), a sanitizer, and a static analyzer (cppcheck / clang-tidy) as callable tools. LLM interprets and explains the raw output.
Phase 4	Code generation prompting	Few-shot prompt templates + optional RAG over internal style guide / libraries for generation task.
Phase 5	Evaluation + iteration	Test all four tasks against real C problems. Only consider light fine-tuning (LoRA) if specific gaps remain after this.

7. Key Reasoning Points

- Existing LLMs already have strong C knowledge from pretraining — no need to train from scratch.
- RAG is used specifically for the Q&A task and for enforcing internal style/standards — it grounds answers in real reference material instead of model memory.
- Analysis and debugging rely on real tool output (compiler, sanitizer, static analyzer) rather than LLM guesswork, because correctness matters and tools are more reliable than reasoning alone.
- Fine-tuning is kept as an optional, later-stage step — only if prompting, RAG, and tool integration together still leave a specific gap in output quality or format.

8. Next Steps

- Finalize which LLM API to use for the base model.
- Curate the initial C knowledge base for the RAG layer (language reference, common pitfalls, style guide).
- Set up gcc + cppcheck/clang-tidy as callable tools.
- Build the router/orchestrator and test each task path independently.
- Evaluate end-to-end and identify any remaining gaps before considering fine-tuning.